

Python Data Structures Practice Notebook

Table of Contents

1. Quick Revision Snapshot
2. Warm-Up Questions
3. Predict the Output
4. Debugging Challenge Zone
5. Edge Cases & Hidden Scenarios
6. Industry-Based Scenarios
7. Think Like an Engineer
8. Hands-On Coding Practice
9. Scenario-Based Coding Tasks
10. Production Thinking Challenges
11. Code Improvement Exercises
12. Interview Preparation
13. Viva Questions
14. Mini Case Study
15. Project Brainstorm
16. Common Mistakes Catalog



Section 1. Quick Revision Snapshot



Key Concepts Quick Reference

Concept	Core Idea
List	Mutable, ordered, allows duplicates. O(1) append/pop from end, O(n) insert/pop from front.
Tuple	Immutable, ordered, allows duplicates. Hashable (if all elements are hashable), so it can be used as a dictionary key.
Dictionary	Mutable key-value store. O(1) average lookup, insert, and delete. Maintains insertion order (Python 3.7+).
Set	Mutable collection of unique elements. O(1) average membership testing, much faster than lists for lookups.
Aliasing	<code>b = a</code> makes both variables reference the same object. Changes through one variable affect the other.

Concept	Core Idea
Shallow vs Deep Copy	<code>.copy()</code> copies only the outer container. <code>copy.deepcopy()</code> recursively copies all nested objects.
Comprehension	<code>[expr for x in iterable if cond]</code> creates a list eagerly. <code>(expr for x in iterable)</code> creates a lazy generator using $O(1)$ memory.
Hashability	Immutable objects like <code>int</code> , <code>str</code> , <code>tuple</code> , and <code>frozenset</code> can be dictionary keys or set members. <code>list</code> , <code>dict</code> , and <code>set</code> cannot.

Important Rules

Master these rules to avoid the most common Python collection bugs.

#	Rule
1	Never use <code>[[]] * n</code> to create independent empty lists — all inner lists reference the same object.
2	Never mutate a list while iterating over it directly. Iterate over <code>list(original)</code> instead.
3	Never use mutable objects (<code>list</code> , <code>dict</code> , <code>set</code>) as dictionary keys or set elements.
4	Always use <code>dict.get(key, default)</code> when a key might not exist.
5	Always use <code>copy.deepcopy()</code> for nested structures that must be modified independently.
6	Membership testing is $O(n)$ for lists but $O(1)$ average for sets and dictionaries.
7	<code>list.append()</code> is $O(1)$ amortized, while <code>list.insert(0, x)</code> is $O(n)$.

Syntax Cheat Sheet

Lists

```

lst = [1, 2, 3]

lst.append(4)           # O(1)
lst.insert(0, 0)        # O(n)

lst.pop()               # O(1)
lst.pop(0)              # O(n)

lst[::-1]               # reversed copy

lst.sort(key=lambda x: -x) # in-place sort
sorted(lst, reverse=True) # new sorted list

```

Tuples

```
t = (1, 2, 3)
```

```
t = (1,) # single-element tuple

a, *rest = (1, 2, 3, 4)

from collections import namedtuple

Point = namedtuple("Point", ["x", "y"])
```

Dictionaries

```
d = {"a": 1, "b": 2}

d.get("c", 0)

d.setdefault("c", []).append(1)

{**d1, **d2}

d1 | d2

d.items()
d.keys()
d.values()
```

Sets

```
s1 = {1, 2, 3}

s1 | s2 # union

s1 & s2 # intersection

s1 - s2 # difference

s1 ^ s2 # symmetric difference

frozenset(s1)
```

Comprehensions

```
[x**2 for x in range(10) if x % 2 == 0]

{x: x**2 for x in range(5)}

{x % 3 for x in range(10)}

(x**2 for x in range(10**8))
```

collections Module

```
from collections import (
    Counter,
    defaultdict,
    deque,
```

```

namedtuple,
ChainMap
)

Counter(["a", "b", "a"]).most_common(1)

defaultdict(int)

deque(maxlen=3)

```



One-Minute Revision Card

Topic	Essential One-Liner
List vs Tuple	List = mutable. Tuple = immutable & hashable.
<code>[[[]] * n]</code> Trap	Creates multiple references to the same inner list.
Dict Lookups	Average O(1) lookup time.
Set Membership	<code>x in my_set</code> is O(1) average.
Shallow Copy	Copies only the outer container.
Generator	Lazy evaluation with O(1) memory.
Counter	Fastest way to count occurrences.
defaultdict(list)	Removes key-existence boilerplate.
Unhashable Error	<code>list</code> , <code>dict</code> , <code>set</code> cannot be dictionary keys.
Dict Ordering	Preserves insertion order since Python 3.7.



Section 2. Warm-Up Questions



Concept Recall Questions

Question:

What is the difference between a list and a tuple, beyond "one is mutable and one isn't"? Why would you ever choose a tuple?



Your Answer:

1.The Iteration over lists is time-consuming and Iteration over tuples is faster. 2.Lists consume more memory and Tuples consume less memory. 3.Lists are more prone to

unexpected changes and errors and Tuples, being immutable, are less error-prone. These are the reasons we will choose tuple over a list.

Question:

Why is dictionary lookup **O(1)** on average? What data structure makes this possible internally?

**Your Answer:**

A dictionary uses a hash table internally and Python computes its hash value using `hash(key)`. The hash determines where the key/value pair should be stored. Looking up a key usually means jumping directly to the correct location instead of searching through all entries. and because of this it doesn't need to scan all elements, lookup is **O(1)** on average.

Question:

What does "**hashable**" mean, and which built-in types are **NOT hashable**?

**Your Answer:**

An object is hashable if it has a hash value which never changes during its lifetime (it needs a `__hash__()` method). (list,dict,set) are not hashable because they are mutable,so their hash could change if their contents changed.

Question:

The difference between `dict.keys()` , `dict.values()` , `dict.items()` and a list? What is Are they **live views** or a **snapshot**?

**Your Answer:**

These keys are live views because they dynamically reflect changes made to the dictionary in real-time. If you add or delete a key-value pair, the view updates automatically without needing to be called again.They are iterable, but not indexable. But, list is static, in-memory copy of the data at the exact moment it was created. Changes made to the original dictionary afterward will not affect the list. Lists are indexable.

Question:

Why is membership testing (`x in ...`) typically much faster for a set than for a list?

**Your Answer:**

A set uses a hash table, while a list stores elements sequentially. In List membership we may need to check each element one by one. While, In list membership it uses hashing to jump directly to the likely location that is the reason it is used.

Question:

What happens in memory when you write: `python b = a` where `a` is a list? Describe or draw the memory relationship.

**Your Answer:**

No new list is created and both variables refer to the same list object.

Question:

What is the difference between a **list comprehension** and a **generator expression** — both syntactically and behaviorally?

**Your Answer:**

List comprehension: Computes everything immediately and Stores all results in memory then produces a list. Generator expression: Produces values lazily (one at a time) and uses much less memory then produces a generator object.

Question:

Name **three things** `collections.Counter` can do that you would otherwise write a manual loop for.

**Your Answer:**

The ``collections.Counter`` can count frequencies, find most common item, Combine or subtract counts.

Rapid-Fire Round

Question:

Which data structure would you use to remove duplicates from a list while preserving order?

Your Answer:

To remove duplicates from a list while preserving order we can use dict.

Question:

Which data structure has guaranteed **O(1)** average lookup by key?

Your Answer:

dict has guaranteed **$O(1)$** average lookup by key

Question:

What error do you get if you try to use a list as a dictionary key?

Your Answer:

TypeError: unhashable type: 'list'

Question:

What is the output of: `python len({1, 1, 2, 2, 3})`

Your Answer:

3

Question:

What method safely retrieves a dictionary value without raising `KeyError` ?

Your Answer:

`get()`

Question:

What is the time complexity of: `python x in my_set` for a set of size `n` ?

Your Answer:

O(1) average

Question:

Which comprehension type uses **curly braces** `{}` and a **colon** `:` ?

Your Answer:

dictionary comprehension

Question:

What is the `namedtuple` field access syntax — `.field` or `["field"]` ?

Your Answer:

`.field`



Fill in the Blanks

1. Complete to safely get a value with a default of 0

`count = my_dict._____("apple", 0)`

Your Answer:

`count = my_dict.get("apple", 0)`

2. Complete to merge two dicts (Python 3.9+ syntax)

`merged = dict_a _____ dict_b`

Your Answer:

`merged = dict_a | dict_b`

3. Complete to create an independent copy of a list of lists

`import copy`
`deep_copy = copy._____(nested_list)`

Your Answer:


```
deepcopy(nested_list)
```

4. Complete to count word frequencies in one line

```
from collections import _____  
word_counts = _____(words)
```

Your Answer:

```
Counter Counter(words)
```

5. Complete to create a dict where missing keys auto-initialize to an empty list

```
from collections import _____  
groups = _____(list)
```

Your Answer:

```
default dict defaultdict
```



Section 3. Predict the Output

Easy

```
In [ ]: '''Predict the output:(1, 2, 3, 1, 2, 3)
          (1, 2, 3, 4)'''
#Explanation:Tuple * repeats elements, and + joins tuples.

t = (1, 2, 3)
print(t * 2)
print(t + (4,))
```

```
In [ ]: '''Predict the output:['a', 'b']
          {'a': 1, 'b': 2, 'c': 3}'''
#Explanation:Keys() returns keys list view, and adding a key updates the dict.

d = {"a": 1, "b": 2}
print(list(d.keys()))
d["c"] = 3
print(d)
```

```
In [ ]: '''Predict the output:3
          {1, 2, 3}'''
```

#Explanation: Sets automatically remove duplicates, so only unique values remain.

```
s = {1, 2, 3, 2, 1}
print(len(s))
print(s)
```

Intermediate

```
In [ ]: '''Predict the output:[1, 2, 3, 4]
          True'''
#Explanation:Both variables point to the same list, so changes affect both.

lst = [1, 2, 3]
lst2 = lst
lst2.append(4)
print(lst)
print(lst is lst2)
```

```
In [ ]: '''Predict the output:{'x': 2, 'y': 3}'''
#Explanation:When merging, later values overwrite earlier ones for the same key.

d1 = {"x": 1}
d2 = {"x": 2, "y": 3}
merged = {**d1, **d2}
print(merged)
```

```
In [ ]: '''Predict the output:[('apple', 3), ('banana', 2)]'''
#Explanation:Counter counts items and returns the most frequent ones in order.

from collections import Counter
words = ["apple", "banana", "apple", "cherry", "banana", "apple"]
print(Counter(words).most_common(2))
```

Tricky

```
In [ ]: '''Predict the output:[[1, 0, 0], [1, 0, 0], [1, 0, 0]]'''
#Explanation:All rows are the same inner list, so changing one changes all.

matrix = [[0] * 3] * 3
matrix[0][0] = 1
print(matrix)
```

```
In [ ]: '''Predict the output:[1]
          [1, 2]
          [1, 2, 3]'''
#Explanation:Default list is reused across calls, so it keeps growing.

def add_item(item, lst=[]):
    lst.append(item)
    return lst
print(add_item(1))
print(add_item(2))
print(add_item(3))
```

```
In [ ]: '''Predict the output:{0: 1, 1: 1, 2: 1}'''
#Explanation:Each value is computed from previous key's value, but d.get(i - 1,

d = {}
for i in range(3):
    d[i] = d.get(i - 1, 0) + 1
print(d)
```

Hidden Behavior

```
In [ ]: '''Predict the output:[1, 3, 5]'''
#Explanation:Removing items from a list while looping skips some elements becaus

my_list = [1, 2, 3, 4, 5]
for item in my_list:
    if item % 2 == 0:
        my_list.remove(item)
print(my_list)
```

```
In [ ]: '''Predict the output:(1, 2, [3, 4, 5])'''
#Explanation:Yes, a is still a tuple, but the list inside it is mutable, so it g

a = (1, 2, [3, 4])
a[2].append(5)
print(a)
```

```
In [ ]: '''Predict the output:{'a': 3, 'b': 2, 'c': 4}'''
#Explanation:Duplicate keys are overwritten, and the last value wins.

keys = ["a", "b", "a", "c"]
values = [1, 2, 3, 4]
result = dict(zip(keys, values))
print(result)
```



Section 4. Debugging Challenge Zone



Bug 1 — Aliasing Trap

```
In [ ]: # This function is supposed to add a discount tag to a list of orders
# WITHOUT modifying the caller's original list. It has a bug – find it.

def apply_discount_tag(orders):
    tagged = orders
    tagged.append("DISCOUNT_APPLIED")
    return tagged
original_orders = ["order_1", "order_2"]
new_orders = apply_discount_tag(original_orders)
print("Original:", original_orders) # Did this change unexpectedly?
print("New:", new_orders)
```

```
# Fix it below: tagged and orders both refer to the same list object, so it also

def apply_discount_tag(orders):
    tagged = orders.copy() # create a new list
    tagged.append("DISCOUNT_APPLIED")
    return tagged

original_orders = ["order_1", "order_2"]
new_orders = apply_discount_tag(original_orders)

print("Original:", original_orders)
print("New:", new_orders)
```

Bug 2 — Mutation During Iteration

```
In [ ]: # This is meant to remove all 'inactive' users from a list. It silently skips so
users = [
    {"name": "Asha", "status": "active"},
    {"name": "Ravi", "status": "inactive"},
    {"name": "Meena", "status": "inactive"},
    {"name": "Karan", "status": "active"},
]
for user in users:
    if user["status"] == "inactive":
        users.remove(user)
print(users) # Expect 2 active users – count what you actually get

# Fix it below: The loop sees "Ravi" and removes it then list shifts left, after
# because of this it is printed.

users = [
    {"name": "Asha", "status": "active"},
    {"name": "Ravi", "status": "inactive"},
    {"name": "Meena", "status": "inactive"},
    {"name": "Karan", "status": "active"},
]

users = [user for user in users if user["status"] != "inactive"]

print(users)
```

Bug 3 — Shallow Copy Trap

```
In [ ]: # A function is supposed to create independent backups of student
# records so the original roster is untouched. It doesn't work as expected.
roster = [{"Asha", [90, 85]}, {"Ravi", [70, 75]}]
backup = roster.copy()
backup[0][1].append(100) # adding a bonus mark to the backup only
print("Roster:", roster) # Did the original get affected too?
print("Backup:", backup)

# Fix it below (hint: which copy function copies ALL levels?):

import copy

roster = [{"Asha", [90, 85]}, {"Ravi", [70, 75]}]
```

```

backup = copy.deepcopy(roster)

backup[0][1].append(100)

print("Roster:", roster)
print("Backup:", backup)

```

Bug 4 — KeyError Crash

```

In [ ]: # This inventory lookup crashes whenever a product ID isn't found.
# Make it crash-proof without changing the overall logic.
inventory = {"SKU101": 50, "SKU102": 12, "SKU103": 0}
def get_stock(sku):
    return inventory[sku]
print(get_stock("SKU101"))
print(get_stock("SKU999")) # <-- crashes here

# Fix it below:

inventory = {"SKU101": 50, "SKU102": 12, "SKU103": 0}
def get_stock(sku):
    return inventory.get(sku, 0) # return 0 if not found

print(get_stock("SKU101"))
print(get_stock("SKU999"))

```

Bug 5 — The N+1 Lookup Problem

```

In [ ]: ## This function matches orders to customer names. It works, but for
# 10,000 orders against 10,000 customers it becomes painfully slow.
# Identify WHY, and rewrite it to be fast at any scale.
orders = [{"order_id": 1, "customer_id": 101}, {"order_id": 2, "customer_id": 102}]
customers = [{"id": 101, "name": "Asha"}, {"id": 102, "name": "Ravi"}]
def attach_customer_name(orders, customers):
    for order in orders:
        for customer in customers: # <-- the problem is here
            if customer["id"] == order["customer_id"]:
                order["customer_name"] = customer["name"]
    return orders
print(attach_customer_name(orders, customers))

# Rewrite using a dict lookup below (should be O(n + m), not O(n * m)):

def attach_customer_name(orders, customers):
    customer_lookup = {
        customer["id"]: customer["name"]
        for customer in customers
    }
    for order in orders:
        order["customer_name"] = customer_lookup.get(order["customer_id"])
    return orders
orders = [
    {"order_id": 1, "customer_id": 101},
    {"order_id": 2, "customer_id": 102}
]
customers = [

```

```

    {"id": 101, "name": "Asha"},
    {"id": 102, "name": "Ravi"}
]

print(attach_customer_name(orders, customers))

```

Bug 6 — Unhashable Type

In [11]: *# This is meant to group transactions by (date, category) and accumulate totals into a dict keyed by that pair. It throws a TypeError. Why, and fix it?*

```

transactions = [
    {"date": "2026-01-01", "category": ["food"], "amount": 200},
    {"date": "2026-01-01", "category": ["food"], "amount": 150},
]
totals = {}
for t in transactions:
    key = (t["date"], t["category"]) # <-- problem
    totals[key] = totals.get(key, 0) + t["amount"]
print(totals)

# Fix it below: The error occurs because dictionary keys must be hashable.

transactions = [
    {"date": "2026-01-01", "category": ["food"], "amount": 200},
    {"date": "2026-01-01", "category": ["food"], "amount": 150},
]
totals = {}
for t in transactions:
    key = (t["date"], t["category"][0])
    totals[key] = totals.get(key, 0) + t["amount"]

print(totals)

```

```
{('2026-01-01', 'food'): 350}
```



Section 5. Edge Cases & Hidden Scenarios



Empty Inputs

```

In [ ]: # What happens with empty containers? Predict, then run.
print(max([], default="no data"))
print(sum([]))
print({}.get("missing", "default"))
print(list({}.items()))

#My Answer:
#no data
#0
#default
#[]

```

Duplicate Keys / Values

```
In [ ]: # Dict literal with duplicate keys – what survives?
d = {"a": 1, "b": 2, "a": 3}
print(d)
# Counter with all-identical items
from collections import Counter
print(Counter(["x", "x", "x"]))

#My Answer:
#{'a': 3, 'b': 2}
#Counter({'x': 3})
```

Deeply Nested Structures

```
In [ ]: # Accessing a deeply nested key that may not exist at any level
data = {"user": {"profile": {"address": {"city": "Pune"}}}}
# This crashes if ANY level is missing. Write a safe accessor function
# that returns None instead of raising, for any depth of missing key.
def safe_get(d, *keys):
    pass # implement this
print(safe_get(data, "user", "profile", "address", "city"))
print(safe_get(data, "user", "profile", "phone")) # should return None, not crash

#My Answer:
def safe_get(d, *keys):
    current = d
    for key in keys:
        if isinstance(current, dict) and key in current:
            current = current[key]
        else:
            return None
    return current
```

Large-Scale Behavior

```
In [17]: import time
# Compare: membership testing in a list of 200,000 items vs a set
big_list = list(range(200_000))
big_set = set(big_list)
start = time.time()
print(199_999 in big_list)
print("list lookup:", time.time() - start, "seconds")
start = time.time()
print(199_999 in big_set)
print("set lookup:", time.time() - start, "seconds")
# Run this and observe the gap – this IS the reason sets exist.

#My Answer:
#List Checks one-by-one, like searching names in a long queue. It has slow member
#Set Uses hashing and jumps directly to the location, like looking up a word in a
```

```
True
list lookup: 0.002742767333984375 seconds
True
set lookup: 5.745887756347656e-05 seconds
```

🚫 Unhashable Elements

```
In [ ]: # Predict what happens, then run each line one at a time:
try:
    s = {1, 2, [3, 4]}
except TypeError as e:
    print("Error:", e)
try:
    d = {[1, 2]: "value"}
except TypeError as e:
    print("Error:", e)
# But a tuple of hashable items works fine:
s2 = {1, 2, (3, 4)}
print(s2)

#My Answer: It is because of the rule that is only immutable objects can be set e
```

➡ Iterator Exhaustion

```
In [ ]: # Generators can only be consumed ONCE. Predict the second print:
gen = (x * x for x in range(5))
print(list(gen))
print(list(gen)) # what do you get the second time? why?

#My Answer: we will get [] as output because There are no values Left and Genera
```



Section 6. Industry-Based Scenarios



Banking

```
In [ ]: # A bank needs to flag accounts with more than 3 transactions over ₹50,000
# in a single day, for fraud review. Use a dict to group by account_id.
'''transactions = [
    {"account_id": "ACC1", "amount": 60000},
    {"account_id": "ACC1", "amount": 70000},
    {"account_id": "ACC2", "amount": 20000},
    {"account_id": "ACC1", "amount": 55000},
    {"account_id": "ACC1", "amount": 51000},
]'''

# Your code: build a dict of account_id -> count of transactions > 50000,
# then list accounts where that count > 3

#My Answer:
transactions = [
    {"account_id": "ACC1", "amount": 60000},
    {"account_id": "ACC1", "amount": 70000},
    {"account_id": "ACC2", "amount": 20000},
    {"account_id": "ACC1", "amount": 55000},
    {"account_id": "ACC1", "amount": 51000},
```



```

]

counts = {}

for t in transactions:
    if t["amount"] > 50000:
        acc = t["account_id"]
        counts[acc] = counts.get(acc, 0) + 1

fraud_accounts = [acc for acc, cnt in counts.items() if cnt > 3]

print(counts)
print(fraud_accounts)

```

Healthcare

```

In [ ]: # A clinic system stores patient allergies as sets so duplicate entries
# never occur. Two records for the same patient need to be merged safely.
'''record_a = {"patient": "P1001", "allergies": {"penicillin", "peanuts"}}
    record_b = {"patient": "P1001", "allergies": {"peanuts", "latex"}}'''
# Your code: merge the allergy sets into one combined, de-duplicated set

#My Answer: set | set → union (new set)
record_a = {"patient": "P1001", "allergies": {"penicillin", "peanuts"}}
record_b = {"patient": "P1001", "allergies": {"peanuts", "latex"}}

merged_allergies = record_a["allergies"] | record_b["allergies"]

print(merged_allergies)

```

E-Commerce

```

In [ ]: # A product catalog has duplicate SKUs from a buggy importer. You need
# to deduplicate by SKU, keeping the FIRST occurrence and preserving order.
''' products = [
    {"sku": "P1", "name": "Mouse"},
    {"sku": "P2", "name": "Keyboard"},
    {"sku": "P1", "name": "Mouse (duplicate import)"},
    {"sku": "P3", "name": "Monitor"},
]'''
# Your code: deduplicate by 'sku', preserving the first occurrence and order
#My Answer:
???
```

HR

```

In [ ]: # HR needs a headcount report: number of employees per department,
# using collections.Counter in a single line.
'''employees = [
    {"name": "Asha", "dept": "Engineering"},
    {"name": "Ravi", "dept": "Sales"},
    {"name": "Meena", "dept": "Engineering"},
    {"name": "Karan", "dept": "HR"},
    {"name": "Divya", "dept": "Engineering"},
]'''
# Your code: build a Counter of department -> headcount

```

```
#My Answer:
from collections import Counter

headcount = Counter(emp["dept"] for emp in employees)

print(headcount)
```

Logistics

```
In [ ]: # A delivery hub needs to track packages in a FIFO + LIFO hybrid: new
# urgent packages go to the front, regular packages go to the back, and
# a max of 5 packages can be held (oldest gets dropped beyond that).
# Use collections.deque with maxlen.
'''from collections import deque
    dock = deque(maxlen=5)'''
# Your code: implement add_regular(package) -> append right
# add_urgent(package) -> append left
# then simulate adding 7 packages (mix of regular/urgent) and print final dock

#My Answer:
from collections import deque

dock = deque(maxlen=5)

def add_regular(package):
    dock.append(package)      # add to the back

def add_urgent(package):
    dock.appendleft(package)  # add to the front

add_regular("P1")
add_regular("P2")
add_urgent("U1")
add_regular("P3")
add_urgent("U2")
add_regular("P4")
add_regular("P5") # exceeds maxlen

print(list(dock))
```



Section 7. Think Like an Engineer



Failure Analysis

For each situation, explain **what will go wrong and why — before fixing anything.**

Question:1

A teammate writes:

```
cache = {}
```

```
def fetch(key, cache={}):  
    ...
```

Six months later, production has a memory leak. Why?

 My Answer:

What goes wrong: the cache keeps growing across all calls to the function, even when the caller thinks they are starting fresh. Why: In Python, default arguments are evaluated once at function definition time, not each time the function is called. So this, creates a single dictionary shared by every call to fetch.

Question:2

A reporting script does:

```
for k in my_dict:  
    del my_dict[k]
```

to clear a dict during iteration.

What happens?

 My Answer:

What goes wrong: the program will raise a runtime error or behave unpredictably. Why: Python dictionaries do not allow structural modification during iteration. This code, changes the size of the dictionary while it is being iterated over.

Question:3

A junior dev uses a list to track "seen" user IDs across 5 million log lines:

```
if uid in seen_list:  
    ...
```

The job that used to take 2 minutes now takes 6 hours. Diagnose this.

 My Answer:

A list requires linear search ($O(n)$) for every lookup and With 5 million log lines, growing seen_list each membership check gets slower over time. First checks: fast-ish and Later checks: scan millions of entries each time. So, Overall complexity becomes roughly: $O(n^2)$ and That's why it explodes from ~2 minutes to ~6 hours.

Question:4

Someone passes a dict as a default argument to a function that's supposed to return a fresh config each time it's called. What's the risk?

 My Answer:

What goes wrong: every function call shares and mutates the same config object.
Why: default arguments are created once. Risk: First call modifies the dict, Second call sees the modified version already there and Changes across calls unintentionally.

Hidden Assumptions

Each statement below contains a flawed assumption. Identify it.

Question:1

"I'll just use a list — order doesn't matter, and lists are simple."

 My Answer:

Assumption: a list is appropriate even when order and indexing are irrelevant, and that simplicity outweighs performance and semantics.

Question:2

"Dicts in Python don't preserve order, so I always re-sort before printing."

 My Answer:

Assumption: Python dictionaries are inherently unordered. But Since Python 3.7+, dicts preserve insertion order as a language guarantee. So sorting is unnecessary.

Question:3

"Sets and lists support the same operations, just sets are faster."

 My Answer:

Assumption: Sets and lists are interchangeable except for performance. But
list:ordered, allows duplicates, supports indexing/slicing, preserves sequence meaning
While, set:unordered, unique elements only, no indexing, optimized for membership tests.

Question:4

"Deep copy is always safer, so I should always use `copy.deepcopy()` instead of `.copy()` just to be safe."

 My Answer:

Assumption:Deeper copying is universally safer without cost or side effects.But `deepcopy()`:recursively copies everything, it can be slow and memory-heavy may break or duplicate unintended shared resources.



Scale Thinking

Each question focuses on performance and system behavior at scale.

Question:1

You have a function that checks `item in my_list` inside a loop that runs **50,000 times**, against a list of **10,000 items**. Roughly how many total comparisons is that, and what's the fix?

 My Answer:

Fix:Replace list with a set. Because Each item in `my_list` is a linear scan. so, this becomes $O(n \times m)$.

Question:2

A `defaultdict(list)` is being used to group **10 million log lines** by `user_id`. Is this the right structure? What could still go wrong at this scale (think: memory)?

 My Answer:

Yes, semantically it fits: grouping values by key is exactly what `defaultdict(list)` is for. But, Memory explosion problem can occur as each log line stored as a full string/object appended into a Python list grouped under potentially many keys.

Question:3

You're asked to make a **"Top 10 most frequent error codes"** report from **1 billion log lines that don't fit in memory**. Would `Counter` alone solve this? What's missing from a pure in-memory approach?

 My Answer:

No. Why: `Counter` is still an in-memory dictionary of all unique error codes.

1. Data doesn't fit in RAM even if each line is processed one-by-one `Counter` still accumulates full state.
2. No external aggregation strategy



Section 8. Hands-On Coding Practice

Beginner

```
In [ ]: # 1. Write a function that takes a list of numbers and returns a NEW list
# containing only the even numbers, using a list comprehension.
'''def get_evens(numbers):
    pass
    print(get_evens([1, 2, 3, 4, 5, 6]))'''

# My Answer:
def get_evens(numbers):
    return [num for num in numbers if num % 2 == 0]

print(get_evens([1, 2, 3, 4, 5, 6]))
```

```
In [ ]: # 2. Write a function that takes a list of words and returns a dict
# mapping each word to its length.
'''def word_lengths(words):
    pass
    print(word_lengths(["python", "ai", "ml"]))'''

# My Answer:
def word_lengths(words):
    return {word: len(word) for word in words}

print(word_lengths(["python", "ai", "ml"]))
```

```
In [1]: # 3. Write a function that returns True if a tuple contains any duplicate values
'''def has_duplicates(t):
    pass
    print(has_duplicates((1, 2, 3))) # False
    print(has_duplicates((1, 2, 2))) # True'''
```

```
# My Answer:
def has_duplicates(t):
    return len(t) != len(set(t))

print(has_duplicates((1, 2, 3)))
print(has_duplicates((1, 2, 2)))
```

False

True

● Intermediate

```
In [ ]: # 4. Given two lists of equal length representing student names and scores,
# build a dict of name -> score, then return the name with the highest score.
'''def top_scorer(names, scores):
    pass
print(top_scorer(["Asha", "Ravi", "Meena"], [88, 95, 91]))'''
```

```
#My Answer:
def top_scorer(names, scores):
    student_scores = dict(zip(names, scores))
    return max(student_scores, key=student_scores.get)

print(top_scorer(["Asha", "Ravi", "Meena"], [88, 95, 91]))
```

```
In [ ]: # 5. Write a function that flattens a list of lists (one level deep)
# using a nested list comprehension.
'''def flatten(list_of_lists):
    pass
print(flatten([[1, 2], [3, 4], [5]])) # [1, 2, 3, 4, 5]'''
```

```
#My Answer:
def flatten(list_of_lists):
    return [item for sublist in list_of_lists for item in sublist]

print(flatten([[1, 2], [3, 4], [5]]))
```

```
In [ ]: # 6. Using collections.defaultdict, group a list of words by their first letter.
'''from collections import defaultdict
def group_by_first_letter(words):
    pass
print(group_by_first_letter(["apple", "ant", "banana", "cat", "car"]))'''
# {'a': ['apple', 'ant'], 'b': ['banana'], 'c': ['cat', 'car']}
```

```
#My Answer:
from collections import defaultdict
def group_by_first_letter(words):
    grouped = defaultdict(list)
    for word in words:
        grouped[word[0]].append(word)

    return dict(grouped)

print(group_by_first_letter(["apple", "ant", "banana", "cat", "car"]))
```

● Advanced

```
In [ ]: # 7. Write a function that finds all pairs of numbers in a list that sum
# to a given target, using a set for O(n) time (not nested loops).
'''def find_pairs_with_sum(numbers, target):
    pass
print(find_pairs_with_sum([2, 7, 11, 15, -2, 9], 9)) # should find (2,7) and

#My Answer:
def find_pairs_with_sum(numbers, target):
    seen = set()
    pairs = set()

    for num in numbers:
        complement = target - num

        if complement in seen:
            # store pair in sorted order to avoid duplicates like (7,2) vs (2,7)
            pairs.add(tuple(sorted((num, complement))))

    seen.add(num)

    return list(pairs)

print(find_pairs_with_sum([2, 7, 11, 15, -2, 9], 9))
# [(2, 7), (11, -2)]
```

```
In [ ]: # 8. Write a function that takes a list of (timestamp, event) tuples and
# returns a dict mapping each event type to a deque of its timestamps,
# keeping only the last 3 occurrences per event (use collections.deque maxlen).
'''from collections import defaultdict, deque
def recent_events_by_type(events, keep=3):
    pass
events = [(1, "login"), (2, "click"), (3, "login"), (4, "login"), (5, "click")]
print(recent_events_by_type(events))'''

#My Answer:??
```

```
In [ ]: # 9. Write a generator function (not a list) that lazily yields running
# totals of a list of numbers – without ever storing the full result list.
'''def running_totals(numbers):
    pass
print(list(running_totals([1, 2, 3, 4]))) # [1, 3, 6, 10]'''

#My Answer:??
```



Section 9. Scenario-Based Coding Tasks

✓ Task 1 — Inventory Reconciliation

```
In [ ]: # Two warehouse systems report stock counts for the same SKUs. Find SKUs
# that DISAGREE between the two systems, and by how much.
'''warehouse_a = {"SKU1": 100, "SKU2": 50, "SKU3": 20}
```



```

warehouse_b = {"SKU1": 100, "SKU2": 45, "SKU4": 10}'''
# Your code:
# 1. Find SKUs present in only one warehouse
# 2. Find SKUs present in both but with different counts, and the difference

#My Answer:
warehouse_a = {"SKU1": 100, "SKU2": 50, "SKU3": 20}
warehouse_b = {"SKU1": 100, "SKU2": 45, "SKU4": 10}

only_in_one = {}
differences = {}

all_skus = set(warehouse_a) | set(warehouse_b)

for sku in all_skus:
    a_val = warehouse_a.get(sku)
    b_val = warehouse_b.get(sku)

    # Present in only one warehouse
    if a_val is None or b_val is None:
        only_in_one[sku] = a_val if a_val is not None else b_val

    # Present in both but different counts
    elif a_val != b_val:
        differences[sku] = abs(a_val - b_val)

print("Only in one warehouse:", only_in_one)
print("Differences:", differences)

```

✓ Task 2 — Customer Segmentation

```

In [ ]: # Segment customers into 'high_value' (spent > 10000), 'medium_value'
# (1000-10000), and 'low_value' (< 1000), using a single pass and a dict
# of Lists.
'''customers = [
    {"name": "Asha", "total_spent": 15000},
    {"name": "Ravi", "total_spent": 500},
    {"name": "Meena", "total_spent": 4500},
    {"name": "Karan", "total_spent": 12000},
]'''

# Your code: build {"high_value": [...], "medium_value": [...], "low_value": [...]}

#My Answer:
customers = [
    {"name": "Asha", "total_spent": 15000},
    {"name": "Ravi", "total_spent": 500},
    {"name": "Meena", "total_spent": 4500},
    {"name": "Karan", "total_spent": 12000},
]
segments = {
    "high_value": [],
    "medium_value": [],
    "low_value": []
}

for customer in customers:
    spent = customer["total_spent"]

    if spent > 10000:
        segments["high_value"].append(customer["name"])

```

```

elif 1000 <= spent <= 10000:
    segments["medium_value"].append(customer["name"])
else:
    segments["low_value"].append(customer["name"])

print(segments)

```

✓ Task 3 — Session Deduplication

```

In [ ]: # A web analytics pipeline logs duplicate page-view events due to a
        # retry bug. Deduplicate using (user_id, page, timestamp) as the uniqueness
        # key, preserving original order, and report how many duplicates were removed.
        '''events = [
            {"user_id": "U1", "page": "/home", "ts": 100},
            {"user_id": "U1", "page": "/home", "ts": 100}, # duplicate
            {"user_id": "U2", "page": "/cart", "ts": 101},
            {"user_id": "U1", "page": "/home", "ts": 105},
            {"user_id": "U2", "page": "/cart", "ts": 101}, # duplicate
        ]'''

#My Answer:

```



Section 10. Production Thinking Challenges



Scale

Question:1

"A function processes a list of 50 million transaction records and does if txn_id in processed_list inside the loop, where processed_list grows with each iteration. Identify the complexity class of this approach overall, and rewrite the approach description (no need to fully code it) using the right structure."

My Answer:

Let, $n=50$ million transactions, $processed_list$ grows from size $0 \rightarrow n$. if $processed_list$ is a list, membership check is $O(k)$ where k is current size of the list. So total cost becomes : $O(1+2+3+\dots+n)=O(n^2)$. Use a hash-based structure (set) instead of a list.



Reliability

Question:2 "A configuration loader does settings = config_dict["env"]["region"] ["timeout"] . In production, this occasionally crashes with KeyError because a partial

config was deployed. Describe two different defensive strategies (without using try/except) that would make this robust, and the tradeoff of each."

 **My Answer:**

??

Readability

Question:3

"Compare these two ways of writing the same logic and discuss in 2-3 sentences which you'd want to maintain in a 6-monthold codebase, and why: Version A result = [y for y in [x*2 for x in data if x > 0] if y < 100] Version B positives = [x for x in data if x > 0] doubled = [x * 2 for x in positives] result = [y for y in doubled if y < 100]"

 **My Answer:**

Version A is more compact but harder to read because it nests two list comprehensions, making it difficult to mentally track the transformation steps. Version B is more verbose but clearly separates filtering and transformation stages into named intermediate variables, which improves readability and debugging. In a 6-month-old codebase, I would prefer Version B because maintainability matters more than brevity—future readers (or you after context is lost) can easily understand and modify each step without unraveling nested logic.



Section 11. Code Improvement Exercises

Refactor 1

```
In [6]: # Refactor this to use a dict comprehension instead of a loop
'''result = {}
    for word in ["python", "java", "go"]:
        result[word] = len(word)'''

# My refactor:
result = {word: len(word) for word in ["python", "java", "go"]}
```

Refactor 2

```
In [7]: # Refactor this O(n^2) duplicate-finder to run in O(n) using a set
'''def has_duplicate(nums):
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] == nums[j]:
                return True
    return False'''

# My refactor:
def has_duplicate(nums):
    seen = set()
    for num in nums:
        if num in seen:
            return True
        seen.add(num)

    return False
```

Refactor 3

```
In [ ]: # Refactor this manual counting loop to use collections.Counter
'''counts = {}
for tag in ["bug", "feature", "bug", "bug", "chore", "feature"]:
    if tag in counts:
        counts[tag] += 1
    else:
        counts[tag] = 1'''

# My refactor:
from collections import Counter

counts = Counter(["bug", "feature", "bug", "bug", "chore", "feature"])
```

Refactor 4

```
In [ ]: # This works for the happy path but crashes on empty input or missing keys.
# Refactor to handle both gracefully.
'''def average_score(students):
    total = 0
    for s in students:
        total += s["score"]
    return total / len(students)
print(average_score([])) # currently crashes - fix it
print(average_score([{"name": "Asha"}])) # currently crashes - fix it'''

# My refactor:
def average_score(students):
    if not students:
        return 0 # or None depending on business logic

    total = 0
    count = 0

    for s in students:
        if "score" in s:
            total += s["score"]
            count += 1
```

```
return total / count if count > 0 else 0
```



Section 12: Interview Preparation

Python Q&A Worksheet

Question 1: What's the difference between `append()` and `extend()` on a list?

 Answer:

`append(x)` adds `x` as a single element (even if it is a list → becomes nested).
`extend(iterable)` iterates over the input and adds each element individually.

Question 2: Why can't lists be dictionary keys?

 Answer:

Dictionary keys must be hashable, which requires immutability. Lists are mutable, so their hash value could change after modification, making them unsafe as keys.

Question 3: What is the output of `{1, 2, 3} == {3, 2, 1}`?

 Answer:

True — sets are unordered collections, so equality depends only on membership, not order.

Intermediate-Level Q&A

Question 4: How would you remove duplicates from a list while preserving order?

 Answer:

`list(dict.fromkeys(my_list))` — dictionaries preserve insertion order (Python 3.7+), so duplicates are removed while order is maintained.

Question 5: What's the time complexity difference between `list.insert(0, x)` and `deque.appendleft(x)`, and why does it matter?

 Answer:

`list.insert(0, x) → O(n)` because all elements must shift. `deque.appendleft(x) → O(1)` because deque is optimized for fast insertions at both ends using block-based linked structure.

Question 6: When would you choose a generator expression over a list comprehension?

 Answer:

Use a generator when you don't need all values in memory at once, such as large datasets or streaming processing. It enables lazy evaluation and improves memory efficiency.

Industry-Level Q&A

Question 7: How would you deduplicate 100 million records that don't fit in memory?

 Answer:

Use external systems or streaming approaches such as external merge sort, database unique constraints, or probabilistic structures like Bloom filters when approximate deduplication is acceptable.

Question 8: Why does `dict.items()` returning a view matter in multi-threaded contexts?

 Answer:

Dictionary views are live and reflect changes in real time. If another thread modifies the dictionary during iteration, it may cause runtime errors or inconsistent results. A copied list avoids this but uses more memory and may be stale.

Question 9: How does Python's dict achieve $O(1)$ average lookup?

 My Answer:

It uses a hash table where keys are hashed into bucket indices. This allows direct access instead of scanning all elements. Collisions are handled using open addressing / probing.



Section 13. Viva Questions

Academic

Question 1: Define mutability and immutability with one example data structure each.

Answer:

Mutable: An object whose contents can be changed after creation. For Example: list.
Immutable: An object whose contents cannot be changed after creation.
Example: tuple.

Question 2: What is the difference between `is` and `==` when comparing two lists with identical contents?

Answer:

`==` compares values/contents and `is` compares object identity (whether both variables refer to the same object in memory).

Question 3: Explain insertion order guarantee in Python dictionaries and since which version it became official language behavior.

Answer:

A Python dictionary preserves the order in which keys are inserted. When you iterate over a dictionary, keys appear in insertion order.
This behavior was an implementation detail in Python 3.6 and it became an official language guarantee in Python 3.7.

Practical

Question 4: Demonstrate (verbally or on paper) what happens in memory when `b = a` is executed for a list `a`, versus `b = a.copy()`.

Answer:

In Case 1 -> `a = [1, 2, 3, 4]` and `b = [1, 2, 3, 4]` Both variables point to the same list object.

In Case 2-> `a=[1, 2, 3]` and `b=[1, 2, 3, 4]` Two separate list objects exist.so, modification affect only b.

Question 5:Walk through how you would detect and fix a "mutation during iteration" bug if you saw one in a teammate's pull request

 Answer:

- 1->Review the loop for operations like: `remove()`, `append()`, `pop()`.
- 2->slice assignment.
- 3->Run a small example and verify expected output.
- 4->Add tests covering edge cases.

In []: *## Application-Based*

Question 6:Give a real scenario (other than the ones in this sheet) where you would choose a set over a list , and justify it using time complexity.

 Answer:

Scenario: Preventing duplicate email sends. Imagine processing millions of email events and checking whether an email address has already received a campaign message.

For millions of lookups:

List → $O(n)$ per lookup

Set → $O(1)$ per lookup

A set dramatically improves performance and naturally prevents duplicates.

Question 7:Give a real scenario where a namedtuple would be preferable to a plain dict, and explain the readability/safety tradeoff.

 Answer:

Scenario: Representing GPS coordinates. Using a dict:`location = {"lat": 28.6139, "lon": 77.2090}`.

Using a namedtuple:

```
from collections import namedtuple
```

```
Location = namedtuple("Location", ["lat", "lon"])
```

```
location = Location(28.6139, 77.2090)
```

For data with a well-defined schema (coordinates, database rows, RGB colors, points, employee records), namedtuple often provides better readability and safety than a plain dictionary.



Section 15. Project Brainstorm

>

- Beginner Projects

- 1. Word Frequency Analyzer — read any text file, use Counter to find the top 10 most common words (excluding common stopwords).
- 2. Contact Deduplicator — given a messy list of contact dicts with inconsistent formatting, deduplicate by normalized phone number.

- Intermediate Projects

- 3. Personal Expense Tracker — store expenses as dicts grouped by category using defaultdict(list) ; support monthly summaries and a "top spending category" report.
- 4. Library Checkout System — use a dict of book_id -> deque to model a hold/waitlist queue per book, with namedtuple records for each checkout.
Resume-Worthy Projects
- 5. Real-Time Log Analyzer — process a streaming log file (simulate with a generator) to detect anomaly bursts (e.g., >100 errors of the same type in 60 seconds), using deque with time-window eviction and Counter for rolling tallies. Showcases generators, deque, and production-style thinking.
- 6. Recommendation Engine (Co-occurrence Based) — given purchase history, build a dict-of-sets co-occurrence map ("customers who bought X also bought Y") and rank recommendations by frequency, entirely with core data structures (no external ML library). Great interview talking point on data-structure tradeoffs at scale.



Section 16. Common Mistakes Catalog



Common Python Mistakes



Beginner Mistakes

1. Shared Inner Lists

```
matrix = [[]] * 3
```

Mistake: Expecting three independent lists.

Issue: All rows reference the same list object.

2. Single-Element Tuple Syntax

```
x = (1)
```

Mistake: Assuming `x` is a tuple.

Correct:

```
x = (1,)
```

3. Unsafe Dictionary Access

```
value = data[key]
```

Mistake: Accessing a key that may not exist.

Better:

```
value = data.get(key, default)
```



Debugging Mistakes

1. Modifying a List While Iterating

```
for item in items:
    if condition(item):
        items.remove(item)
```

Issue: Elements may be skipped or produce unexpected behavior.

2. Assuming `.copy()` Performs a Deep Copy

```
copy_data = original.copy()
```

Issue: Only the top-level structure is copied.

Use:

```
import copy
copy_data = copy.deepcopy(original)
```

3. Confusing `is` and `==`

```
a is b
```

Use `is` for identity checks.

```
a == b
```

Use `==` for value comparisons.



Design Mistakes

1. Using a List Instead of a Dictionary

Mistake: Performing frequent lookups in a list.

Better: Use a dictionary for $O(1)$ average lookup time.

2. Mutable Default Arguments

```
def f(x, cache={}):  
    ...
```

Issue: State persists across function calls.

Better:

```
def f(x, cache=None):  
    if cache is None:  
        cache = {}
```

3. Nested Loops for Dataset Joins

```
for a in data1:  
    for b in data2:  
        ...
```

Issue: $O(n \times m)$ complexity.

Better: Build a dictionary index and achieve $O(n + m)$.



Industry-Level Mistakes

1. Loading Massive Datasets into Memory

Mistake:

```
data = list(read_large_file())
```

Issue: Memory exhaustion.

Better: Stream data using generators.

2. Using a Dictionary as a Database

Mistake: Relying on an in-memory dictionary for growing datasets.

Issue: Scalability and persistence limitations.

3. Mutating Shared Global State

CACHE = {}

Issue: Multiple modules modifying shared state can create difficult-to-debug behavior.

Better: Define clear ownership and encapsulate state management.

Key Takeaway

Most Python bugs come from:

- Hidden mutability
- Incorrect assumptions about copying
- Choosing the wrong data structure
- Ignoring scalability constraints

Understanding these patterns prevents many real-world production issues.